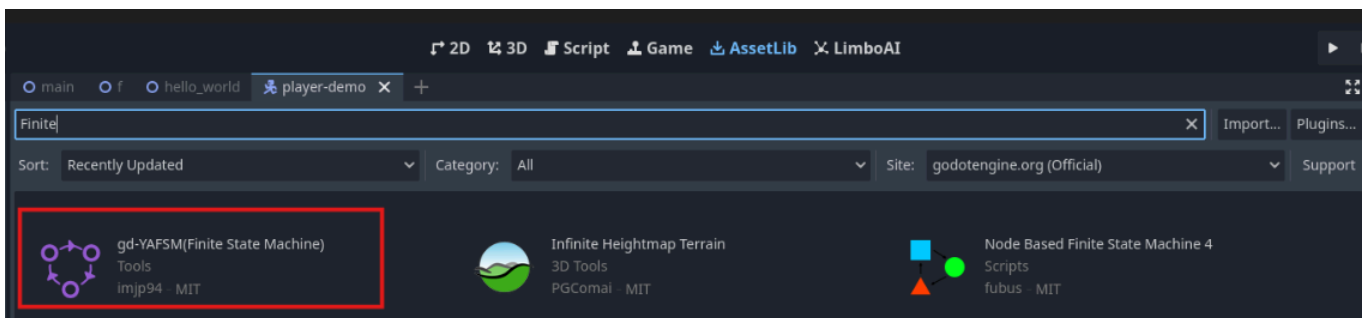
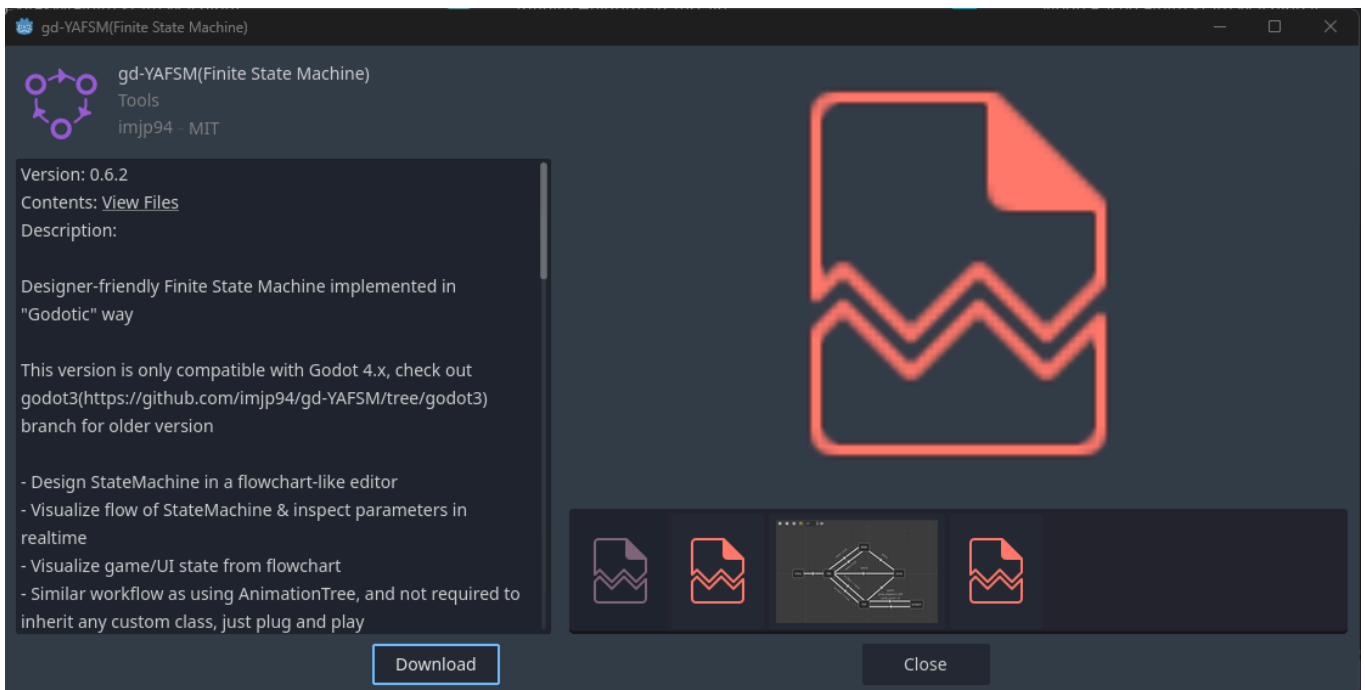


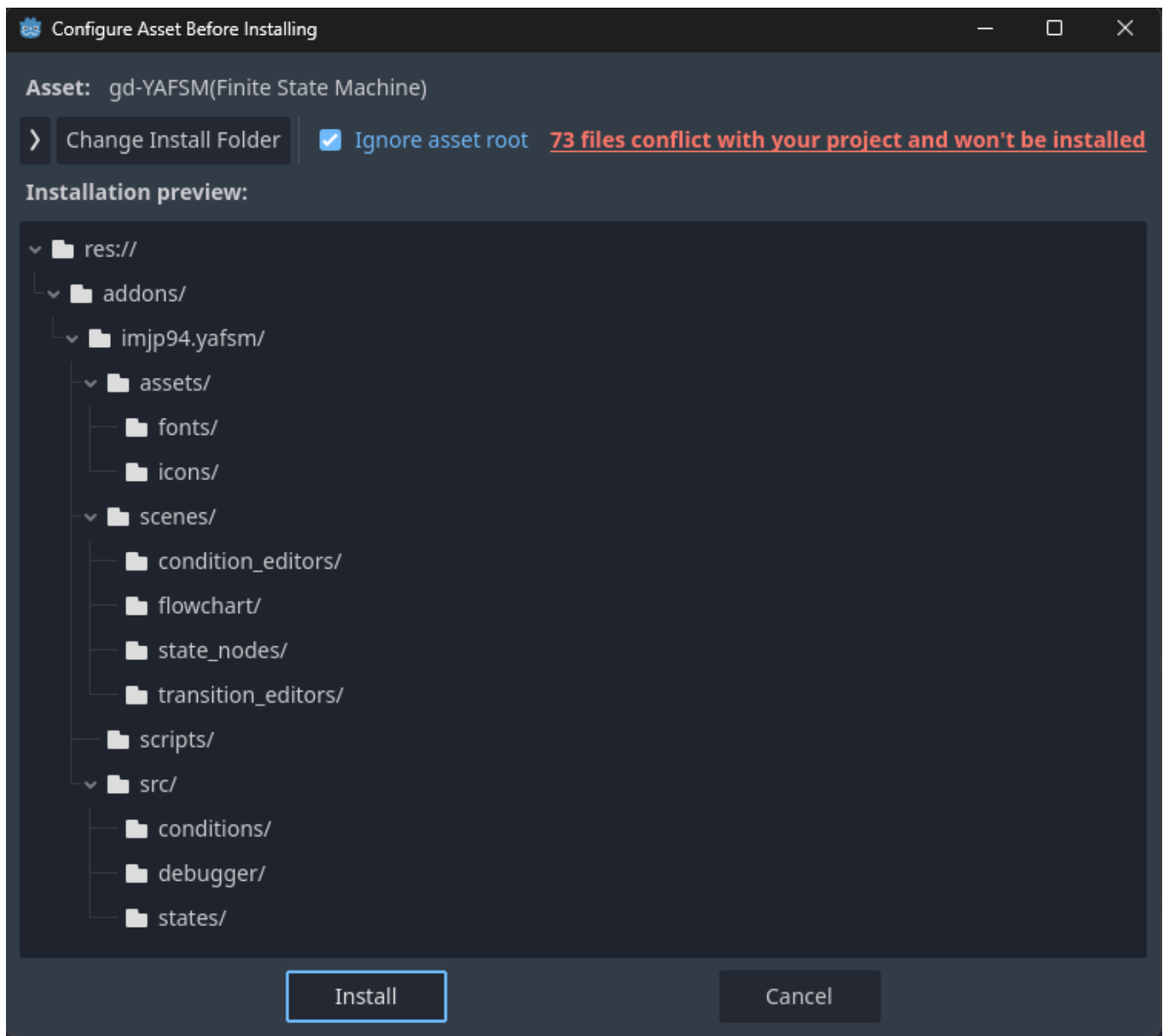
Platformer 2 - Character Controllers and Finite State Machines

Godot's basic movement script for the CharacterBody2D node provides many of the basic functions we might need a player controlled entity to have. However, we will often have complex interactions that we may want to organize and control transitions between. To do this we can use a *Finite State Machine*. Godot doesn't have a built in Finite State Machine but its included Asset Library has many available to use. For this assignment we will be using the "gd-YAFSM(Finite State Machine)" addon. You can find this in the AssetLib tab by searching "Finite State Machine":

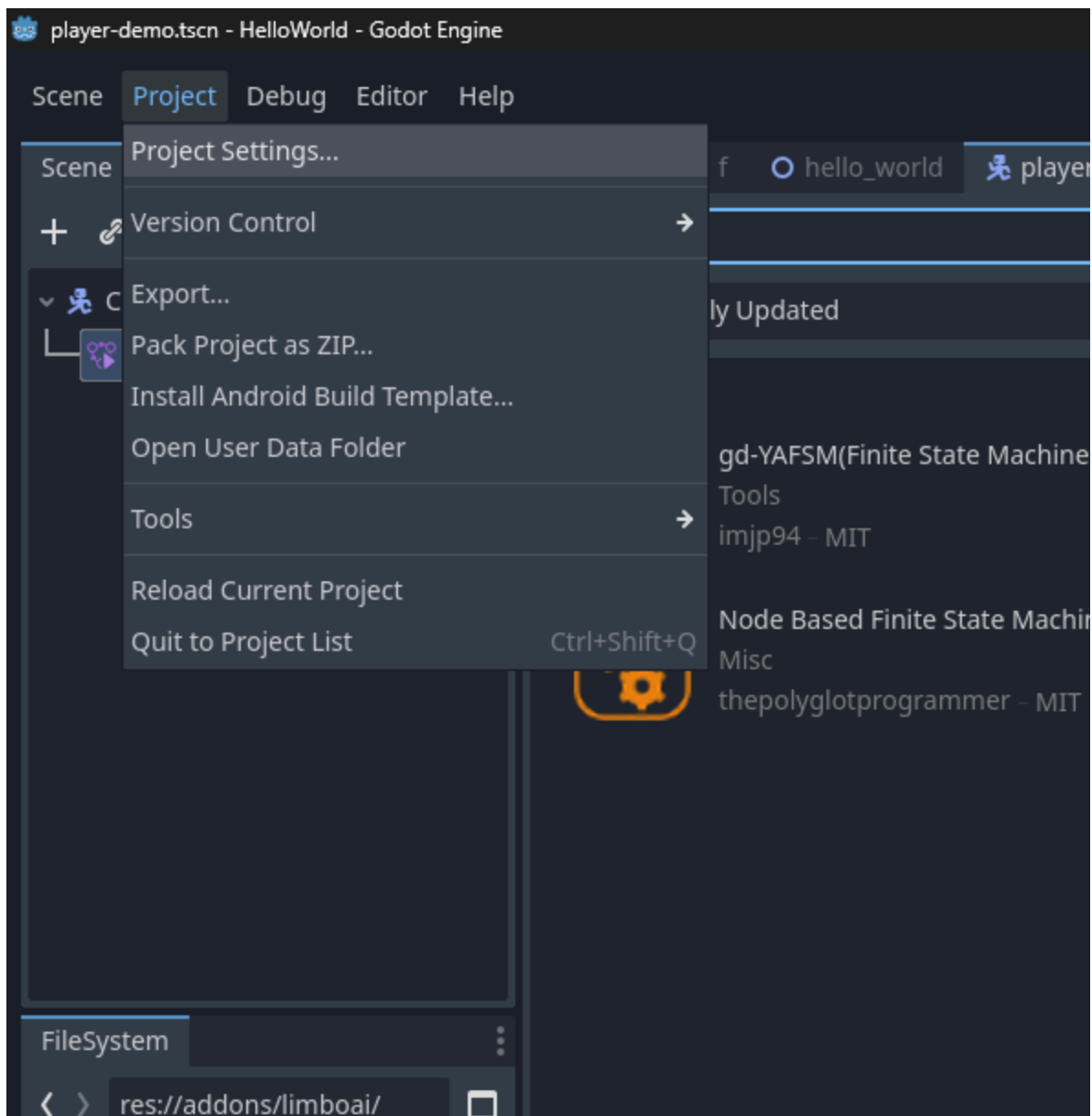


We can click on this and press "Download" and then "Install" to add it to our project:

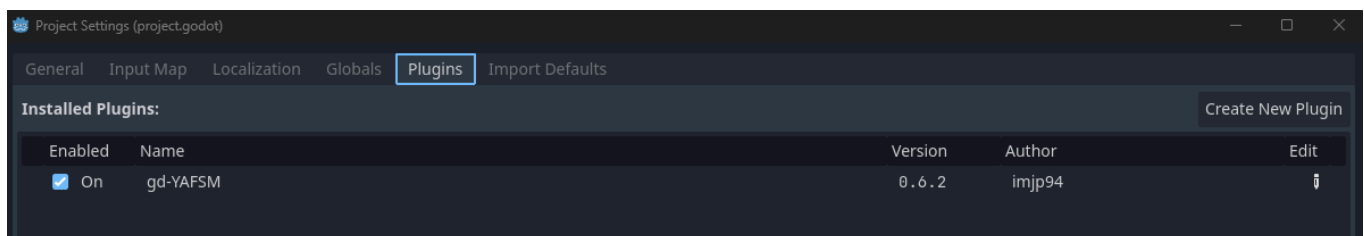




The last thing we need to do is make sure the plugin is active. We'll do this in the "Project Settings" menu found under the "Project" dropdown menu:



Then, in the "Plugins" tab, we'll make sure that the gd-YAFSM plugin is enabled:



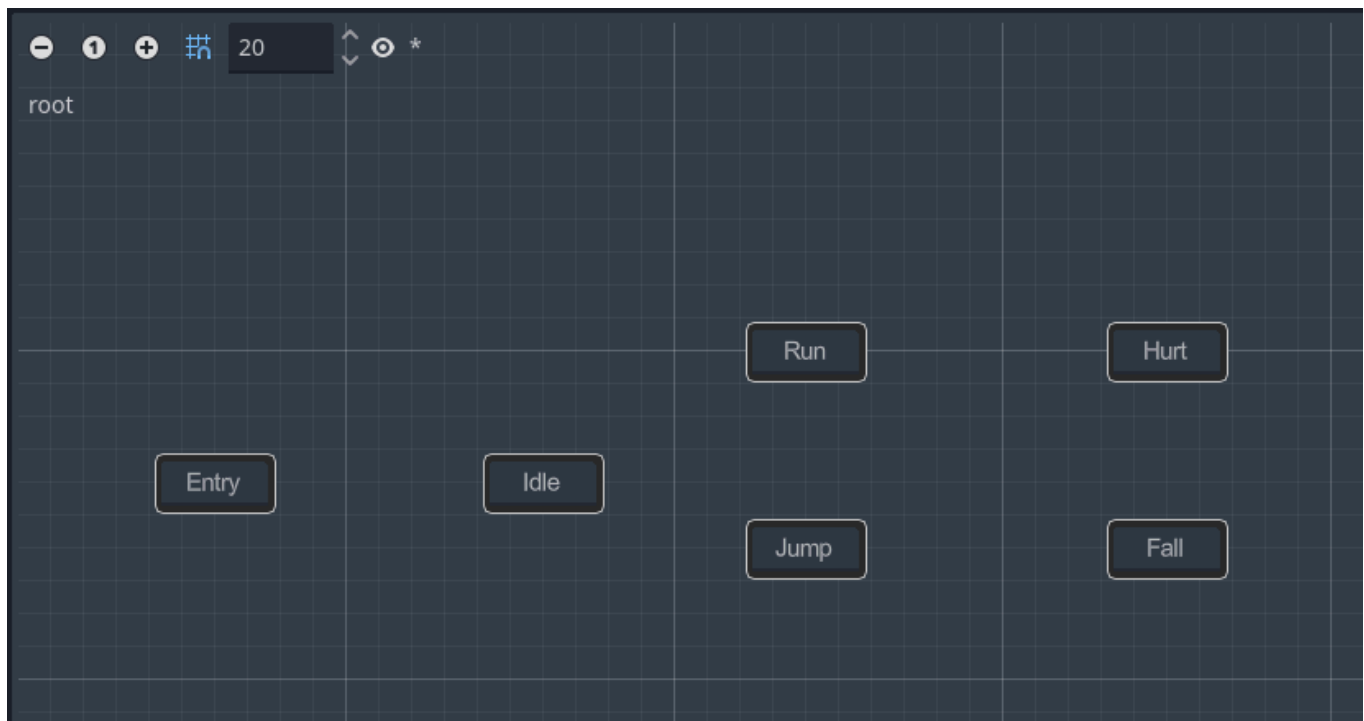
Now let's talk more about what a Finite State Machine is.

Finite State Machines: States and Transitions

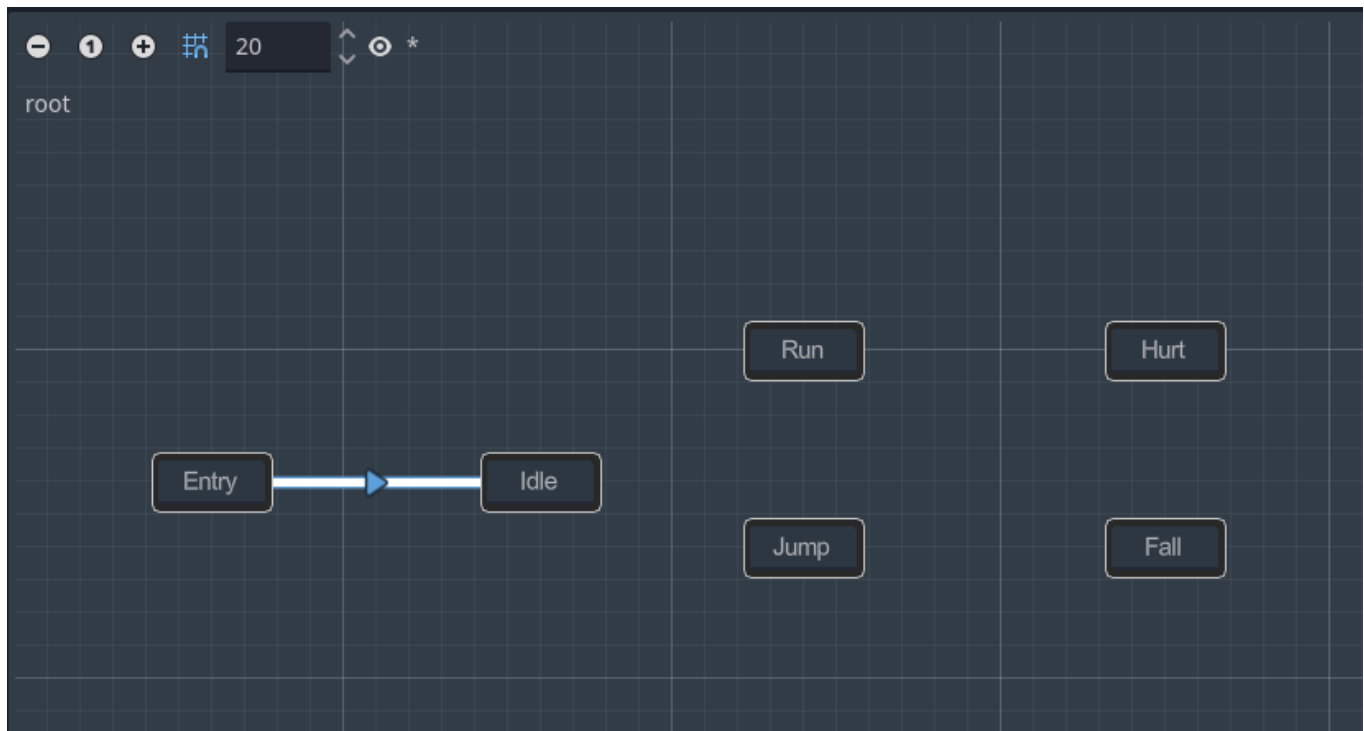
A State is a discrete set of instructions that an entity will follow. An entity can only be in one State at a time, though they can have as many States as are needed for the behaviors of the entity. For example, if we are defining the states of a "Player" entity we may have the following states with the attached behaviors:

- Idle
 - In this state the player is not moving
 - Plays the "idle" animation
- Run
 - Player can move left and right using the A and D keys
 - Plays the "run" animation
- Jump
 - Propels the player upward
 - Player can move left and right using the A and D keys
 - Plays the jump animation
- Fall
 - The player can move left and right using the A and D keys
 - Plays the Fall animation
- Hurt
 - Reduces the player's health by 10
 - Pushes the player away from the source of damage
 - Stops the player from taking more damage while in the state

The gd-YAFSM State Machine for these states would look like this:



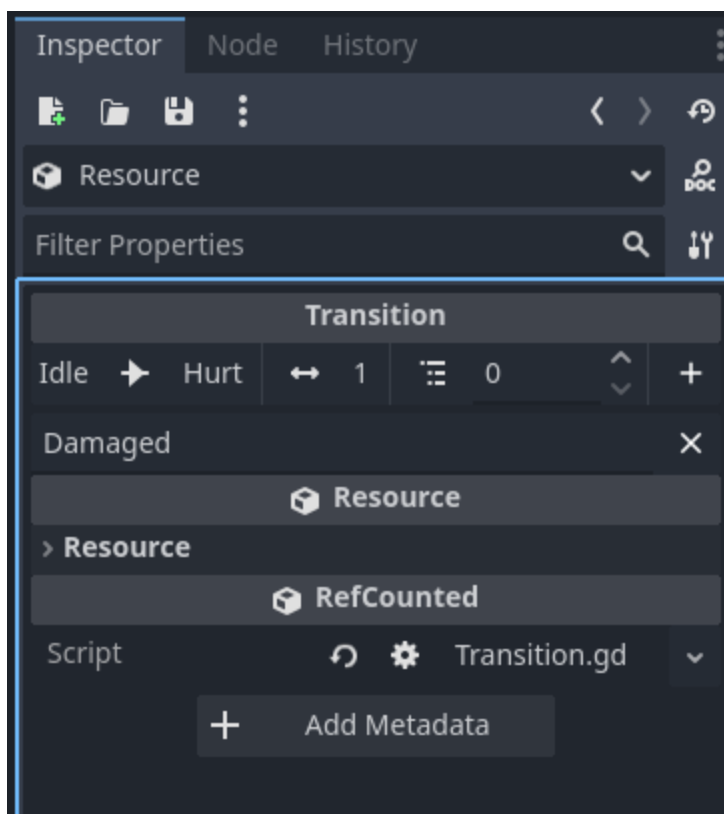
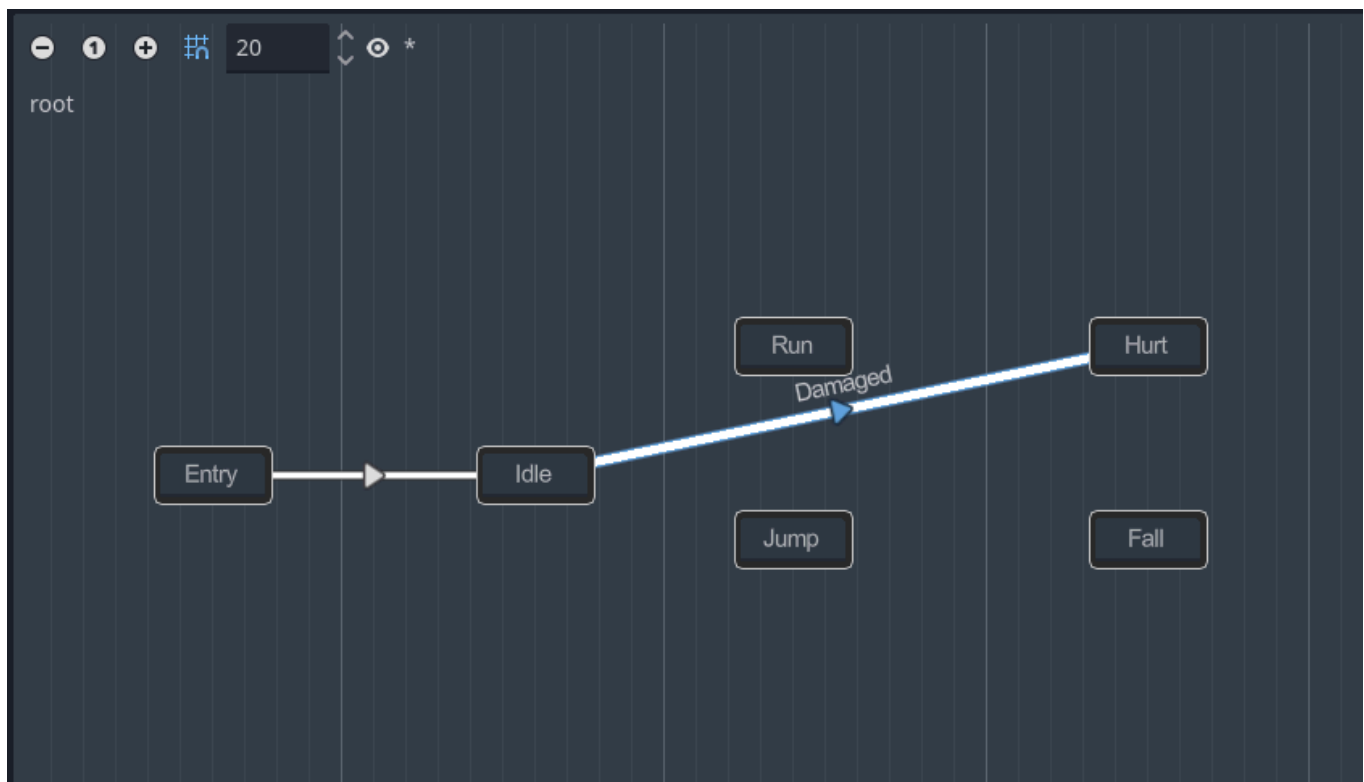
We control which state is currently active using *transitions*. The "Entry" state is always the first state. We can add a *transition* to another state by holding the shift key and clicking-and-dragging from the source state (Entry in this case) to its destination state (Idle in this case):



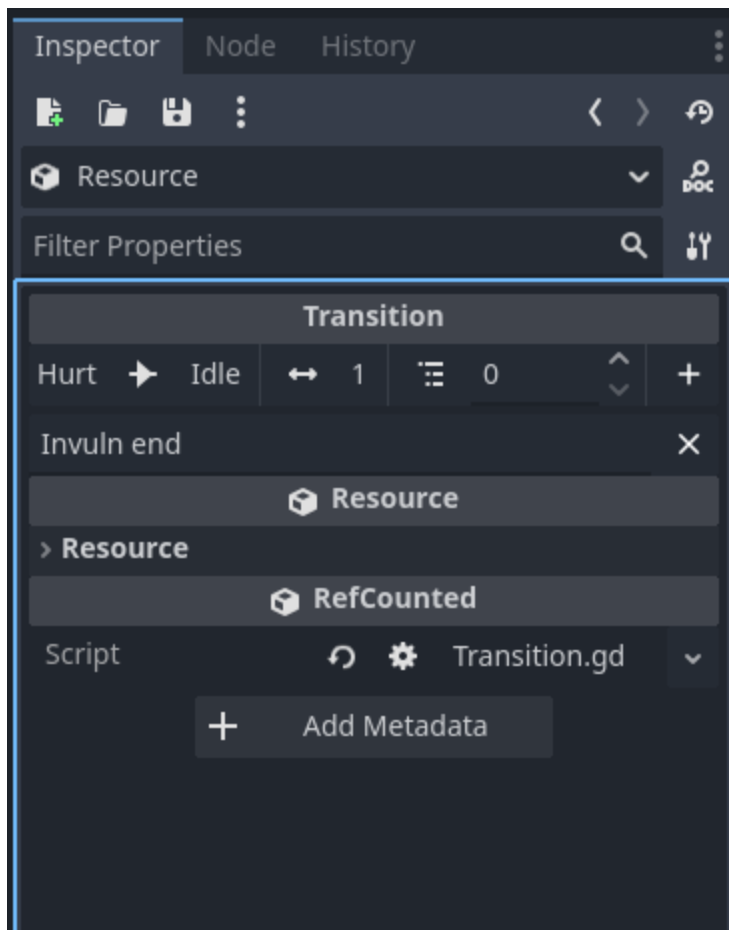
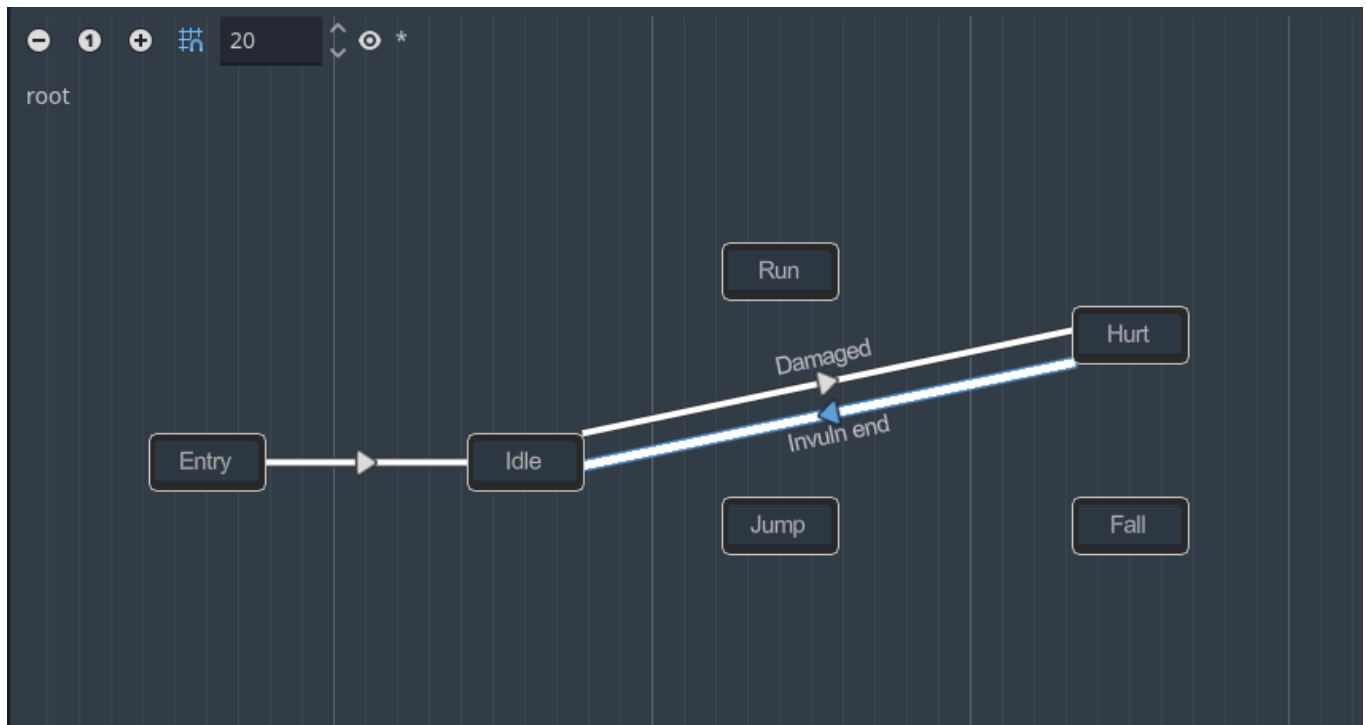
This transition will happen automatically when the game starts as there are no *conditions* to stop the transition. If we were to connect all of these States without any conditions we would infinitely cycle through every state without staying in a single one. To stop this from happening we can add conditions to the transitions. We can add a few different kinds of conditions, here are the most common:

- Trigger
 - Allows the transition to happen when something such as a Button press happens
- Float
 - Compares a parameter (we'll talk about these later) to a value and allows the transition when the comparison is true
- Boolean
 - Takes a parameter that is either True or False and allows the transition when the value is True

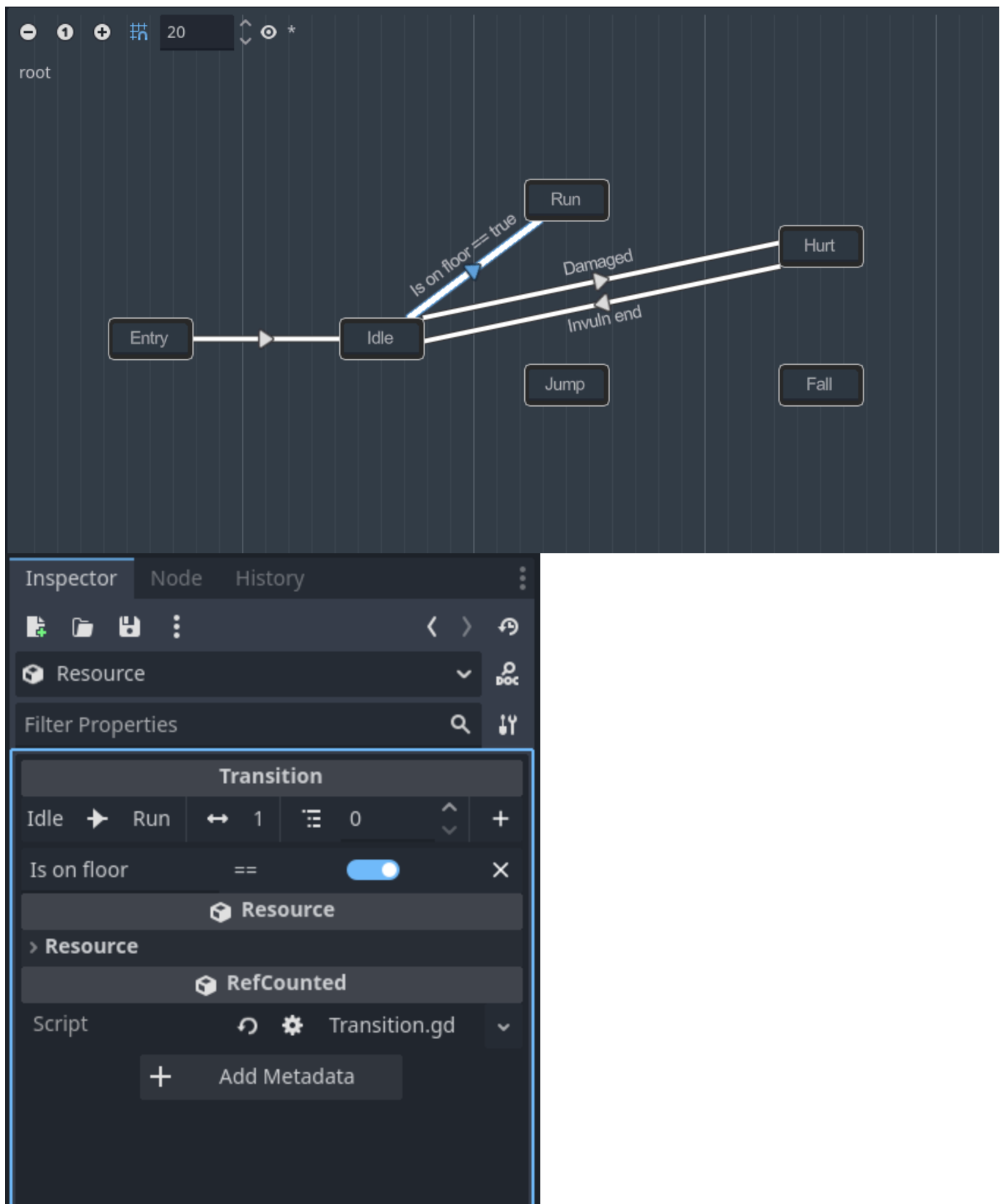
Let's define a new transition here, from the Idle state to the Hurt state. We need to be able to transition to this state right when the player takes damage so we will use the "Trigger" transition condition. We'll make the name of the Trigger to "Damaged"



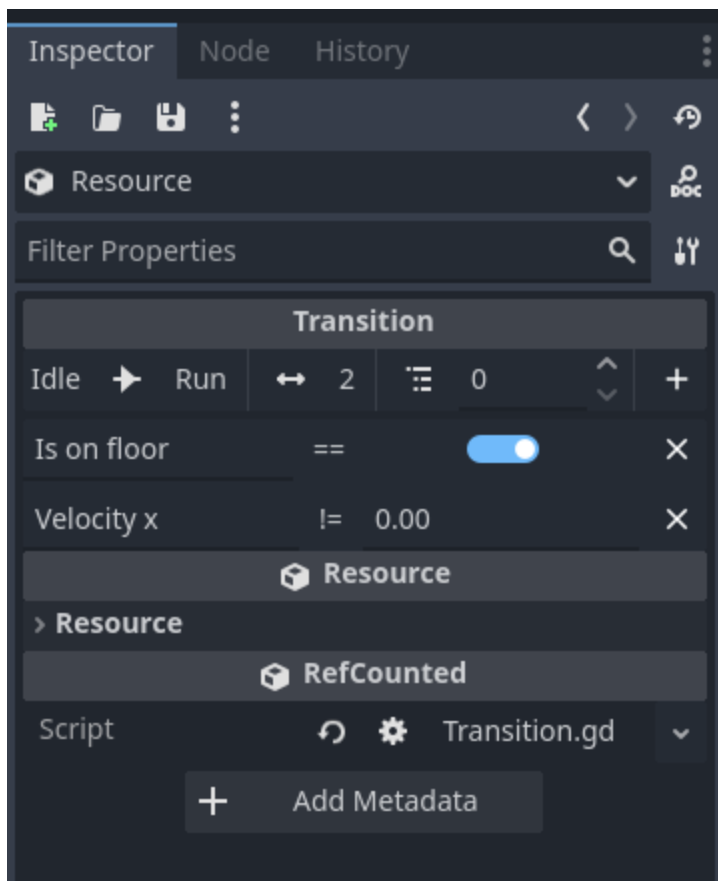
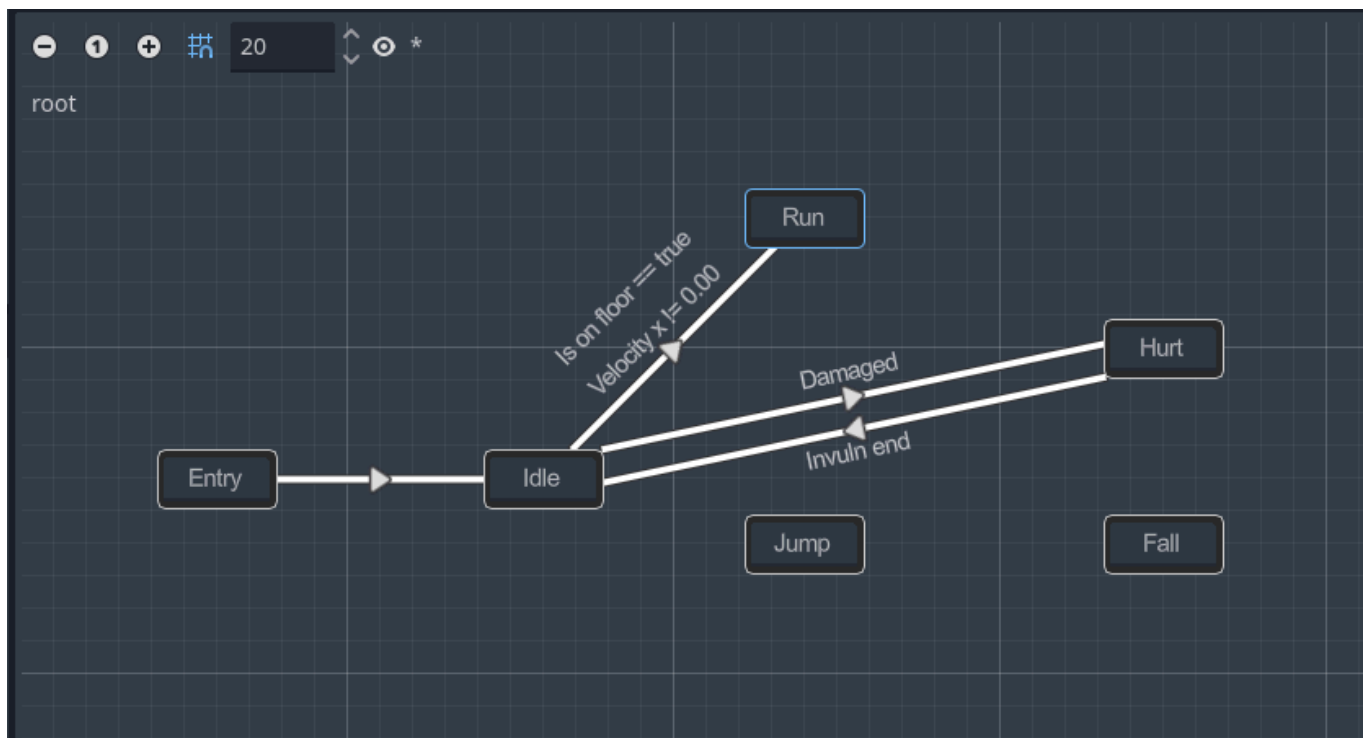
We then need to determine when to transition back to the Idle state. We'll add a second transition called "Invuln end"



Next, let's tackle the transition to our Run state. We will need to have *two* conditions for this transition, as we only want to enter our Run state if our player is currently on the ground. For this we'll add a Boolean condition called "Is on floor" and set it to allow the transition when set to True:



We can then add another transition, this time a float transition. We'll call this parameter "Velocity x" and set it to allow the transition when it is not equal to 0.



State Logic

Now that we can transition between states let's set up our code to handle the *logic* for our states. We will do this in the script attached to our CharacterBody2D. We'll need to do some setup code first. We'll need to first get a reference to our StateMachinePlayer Node:

```
extends CharacterBody2D
```

```
@onready var state_machine_player: StateMachinePlayer = $StateMachinePlayer
```

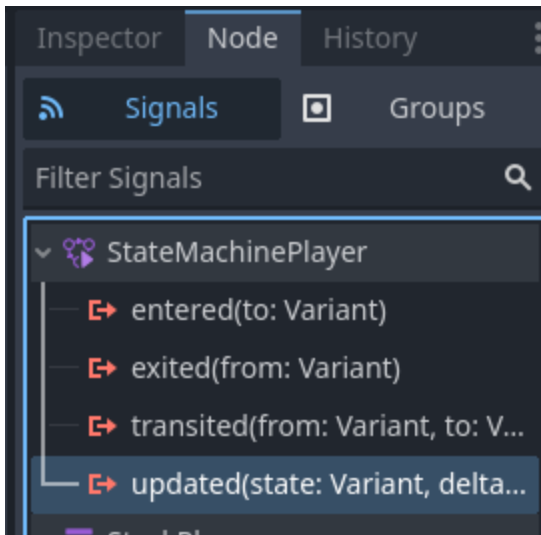
Then, we'll need to add a variable to hold the *active* state, we'll set it to "Idle" by default:

```
extends CharacterBody2D
```

```
@onready var state_machine_player: StateMachinePlayer = $StateMachinePlayer
```

```
var active_state = "Idle"
```

Next, we'll connect the "updated" signal from our StateMachinePlayer to our Player script:



```
extends CharacterBody2D
```

```
@onready var state_machine_player: StateMachinePlayer = $StateMachinePlayer
```

```
var active_state = "Idle"
```

```
... # There is extra code here that is omitted for readability
```

```
func _on_state_machine_player_updated(state: Variant, delta: Variant) -> void:  
    pass # Replace with function body.
```

Last thing we need to do is to add some code to this signal function that sets the `active_state` variable to the state argument's value:

```
extends CharacterBody2D
```

```
@onready var state_machine_player: StateMachinePlayer = $StateMachinePlayer

var active_state = "Idle"

... # There is extra code here that is omitted for readability

func _on_state_machine_player_updated(state: Variant, delta: Variant) -> void:
    active_state = state
    pass # Replace with function body.
```

State Logic: State Functions

Next, we will define functions for every *state* that our entity has. In this case we'll have the following functions:

```
func idle()

func run()

func jump()

func hurt()

func fall()
```

We can then add code to each function to define the *behavior* in each state. Here's what our run function might look like:

```
func run():
    var direction = Input.get_axis("ui_left", "ui_right")

    if direction:
        velocity.x = direction * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED)
```

State Logic: Updating the State Machine Parameters

Now that we have our state logic set, we need to update the state machine's parameters to control the transitions. We will use the following methods() of the StateMachinePlayer to update it:

- Triggers

- `StateMachinePlayer.set_trigger("Trigger Name")`
- Floats and Booleans
 - `StateMachinePlayer.set_param("Parameter Name", value)`

Some parameters we will need to update every frame, while others will be updated only occasionally. We'll place any that need to be updated every frame (such as the "Velocity x" parameter) in the `_physics_process()` function like this:

```
func _physics_process(delta: float) -> void:
    StateMachinePlayer.set_param("Velocity x", velocity.x)
```

State Logic: Running the state

In order to run our states we need to add a bit of code to our process function. We will need to check what the active state is and then run the proper function. We will do this by using a *match* statement. Think of this as a powerful if statement.

Below is an example of what the match statement for the above states would look like:

```
match active_state:
    "Idle":
        idle()
    "Run":
        run()
    "Jump":
        jump()
    "Fall":
        fall()
    "Hurt":
        hurt()
```

Now it's your turn!

In your platformer project complete the following tasks:

1. Add a `StateMachinePlayer` Node to your Player Scene
2. Add the following states to the `StateMachinePlayer`'s `StateMachine`
 1. Idle
 2. Run
 3. Jump
 4. Fall
 5. Hurt

3. Add the following transitions and conditions to the StateMachine:

1. Idle

1. To Run when the player is on the ground and moving
2. To Jump when the player is on the ground and presses the "ui_accept" input
3. To Fall when the player is *not* on the ground
4. To Hurt when the "Hurt" trigger is set

2. Run

1. To Idle when the player is on the ground and not moving
2. To Jump when the player is on the ground and presses the "ui_accept" input
3. To Fall when the player is *not* on the ground
4. To Hurt when the "Hurt" trigger is set

3. Jump

1. To Idle when the player is on the ground
2. To Fall when the player's y velocity is positive
3. To Hurt when the "Hurt" trigger is set

4. Fall

1. To Idle when the player is on the ground
2. To Hurt when the "Hurt" trigger is set

5. Hurt

1. To Idle when the "Invuln end" trigger is set

4. Add functions for each state to the Player script, give them the following logic. If the logic is already in the process() function move it into the state function and remove it from the process() function. Otherwise, leave it there:

1. Idle

1. Set the "Jump" trigger when you press the jump button

2. Run

1. Move the player to the left and right when the "ui_left" and "ui_right" buttons are pressed
2. Set the "Jump" trigger when you press the jump button

3. Jump

1. Sets the player's y velocity to a negative value once at the beginning of the state
2. Move the player to the left and right when the "ui_left" and "ui_right" buttons are pressed

4. Fall

1. Move the player to the left and right when the "ui_left" and "ui_right" buttons are pressed

5. Hurt

1. Print "ouch"
2. Set the "Invuln end" trigger
5. Add a print() method to your _physics_process() function that prints the current state
6. Create a test scene, make sure your player can move between every state